High-Level Development Guide

1. Delivery strategy

Implement this in parallel with the current non-AI flow, not as a full replacement on day one.

Recommended rollout:

1. keep current endpoints stable
2. introduce LangGraph modules behind new services
3. run the graph in shadow mode against stored events
4. compare graph outputs against the current manual/tester flow
5. enable auto-push only after validation quality is acceptable

2. Phase 1: Extract the orchestration boundary

Goal:

- stop growing `jenkins_service.py`

Tasks:

- add a new graph package
- define `GraphState`
- add a service that loads a stored event and converts it to graph input
- keep the webhook endpoint, but hand off to a graph runner instead of calling `_analyze_event()` directly

Suggested rule:

- do not remove the current sync analyzer until the graph path is proven

3. Phase 2: Build Node 01 against real Trivy output

Goal:

- parse the real payload already present in `requests.jsonl` and `dev_test/notebook/trivy_scan_input.txt`

Tasks:

- build a table parser for Trivy console output
- normalize findings into `VAFinding`
- support repeated lines where one package spans several CVEs
- capture all fixed versions as a list

Acceptance signal:

- the real stored sample should no longer end in `supported = false`
- the parser should emit multiple findings and a meaningful issue summary

4. Phase 3: Build Node 02 repo fetch and manifest planning

Goal:

- fetch source manifests and produce BOM-friendly remediation candidates

Tasks:

- wrap current `fetch_analysis_files()` in a cleaner repo-reader service
- add parsers for Maven and Gradle control points
- detect parent POM, imported BOMs, `dependencyManagement`, Gradle platform usage, and property-managed versions
- add Dockerfile parsing for base image and package-manager upgrades

Important design choice:

- Node 02 should produce candidate plans, not final approved changes

5. Phase 4: Build Node 03 validation

Goal:

- create a hard gate before any branch write

Tasks:

- validate each candidate against manifest ownership rules
- collapse duplicate or conflicting changes
- mark plans as auto-safe, low-confidence, or manual-review-required

Acceptance signal:

- a direct dependency version change must be rejected if a parent/BOM/property owns that version
- one approved plan should explain why it is the chosen BOM-friendly strategy

6. Phase 5: Build Node 04 GitHub multi-file writer

Goal:

- write a full approved change set, not just one file

Tasks:

- create a `github_writer` that can commit several files in one branch update
- keep compare URL / PR creation behavior similar to the current flow
- store branch result back into the run record

Recommended implementation detail:

- use GitHub Git blobs/tree/commit APIs or a well-scoped sequence of contents API updates
- treat branch creation and PR creation as separate steps with explicit error reporting

7. Phase 6: Observability and operator UX

Goal:

- make graph execution inspectable

Tasks:

- store node outputs by run
- expose run status endpoints
- show node-by-node progress in the dashboard
- preserve raw payload, extracted issue list, chosen remediation plan, validation notes, and push result

8. Data persistence guidance

Short term:

- reuse the current JSONL event store pattern for experiments

Better next step:

- split event storage from graph-run storage
- add a persistent run/checkpoint store for node outputs and retries

If LangGraph checkpointing is introduced, keep event metadata and graph state records clearly linked by `event_id` and `run_id`.

9. Testing guidance

Prioritize tests in this order:

1. Node 01 parsing tests against full Trivy samples
2. manifest parser tests for Maven/Gradle/Docker variations
3. Node 03 validation tests for BOM-owned dependencies
4. Node 04 GitHub writer tests for multi-file commits and existing-branch reuse
5. API integration tests for webhook to graph-run creation

Critical regression scenarios:

- vulnerability table with many rows
- package with multiple fixed versions
- parent-managed Spring Boot app
- imported BOM app
- multi-module Maven repo
- Gradle platform-managed app

- Docker-only remediation case
- mixed `pom.xml` plus `Dockerfile` remediation case

10. Suggested first implementation target

Best first slice:

1. webhook stores event
2. Node 01 parses Trivy sample into structured findings
3. Node 02 fetches repo files
4. Node 03 marks plan as manual-review-required if full BOM logic is not ready
5. Node 04 stays disabled

That gives you visible AI progress early without risking bad automated commits.